Mini Project

Scientific Calculator with DevOps CI/CD Pipeline

CS 816: Software Production Engineering

A Project Report Submitted
in Partial Fulfilment of the Requirements
for the Award of the Degree

MASTER OF TECHNOLOGY

in

Computer Science and Engineering

Submitted by

Abhijeet Kumar Gupta

(MT2025003)

Submitted to
Department of Computer Science and Engineering
International Institute of Information Technology
Bangalore - 560100, India

Abstract

In modern software development, automation and continuous integration have become essential for delivering reliable and high-quality applications. DevOps is a methodology that combines software development and IT operations to automate different stages of the software lifecycle such as building, testing, deployment, and monitoring.

In this project, a Scientific Calculator application is developed and integrated with a DevOps based CI/CD pipeline. The calculator is implemented as a menu-driven command line interface (CLI) program that performs various mathematical operations including addition, subtraction, multiplication, division, percentage calculation, square root, factorial, natural logarithm, and power functions.

The source code of the application is maintained using GitHub for version control. Whenever code changes are pushed to the repository, Jenkins automatically triggers the CI/CD pipeline to build and test the application using Maven. After successful testing, a Docker image of the application is created and pushed to Docker Hub for containerized deployment. Ansible is used to automate the deployment process on the target environment, ensuring consistency and repeatability.

This project demonstrates how DevOps tools can be integrated to automate the development and deployment process. By implementing continuous integration and continuous deployment, the system improves efficiency, reduces manual errors, and enables faster and more reliable software delivery.

1. Introduction

In modern software development, automation and continuous integration have become essential for delivering reliable and high-quality applications. DevOps is a methodology that combines software development and IT operations to automate processes such as building, testing, deployment, and monitoring of applications.

In this project, a Scientific Calculator application is developed and integrated with a DevOps-based CI/CD pipeline. The application is a menu-driven command line interface (CLI) program that performs various mathematical operations such as addition, subtraction, multiplication, division, percentage calculation, square root, factorial, natural logarithm, and power functions.

To automate the software development lifecycle, several DevOps tools are used including GitHub, Maven, Jenkins, Docker, Docker Hub, and Ansible. GitHub is used for version control and source code management. Maven is used for building and managing the project dependencies. Jenkins automates the build and testing process through a CI/CD pipeline. Docker is used to containerize the application, while Docker Hub is used to store and manage the Docker images. Ansible is used for automated deployment and configuration management.

The main objective of this project is to demonstrate how different DevOps tools can be integrated to automate the development, testing, and deployment processes, making software delivery faster, more reliable, and efficient.

---

2. Objectives

The main objectives of this project are:

- To develop a command-line based scientific calculator application.
- To implement unit testing for validating the calculator operations.
- To automate the build process using Maven.
- To create a CI/CD pipeline using Jenkins.
- To containerize the application using Docker.
- To push the Docker image to Docker Hub.
- To deploy the application automatically using Ansible.
- To monitor application logs for basic monitoring and analysis.

---

3. DevOps and its Importance

DevOps is a modern software development approach that integrates development and operations teams to improve collaboration and automate the software development lifecycle. It focuses on continuous integration, continuous deployment, automated testing, and monitoring to deliver software more efficiently and reliably.

The main advantages of DevOps include:

• **Speed:** DevOps enables faster development and deployment of applications by automating various stages of the software lifecycle.

• **Automation:** Build, testing, and deployment processes are automated, which reduces manual effort and minimizes human errors.

• **Reliability:** Automated testing and deployment ensure that the software remains stable and performs consistently across different environments.

• **Continuous Integration and Deployment:** Code changes are automatically integrated, tested, and deployed, which reduces delays and allows faster delivery of software updates.

---

4. Technology Stack

The following tools were used in this project as part of the DevOps pipeline.

Stage	Tool Used	Purpose
Source Control	GitHub	Stores the source code and manages version history
Testing	JUnit	Performs unit testing of calculator functions
Build Tool	Maven	Compiles the source code and manages project dependencies

CI/CD	Jenkins	Automates the build and deployment pipeline
Containerization	Docker	Packages the application and its dependencies into containers
Image Registry	Docker Hub	Stores and distributes Docker images
Configuration Management	Ansible	Automates the deployment process
Monitoring	Log Files	Stores application logs for monitoring and debugging

---

5. System Architecture

The system architecture describes how different DevOps tools interact to automate the development, testing, and deployment process of the scientific calculator application. The entire workflow is automated using a CI/CD pipeline.

The workflow of the system is as follows:

• The developer writes the source code for the scientific calculator application.

• The source code is pushed to the GitHub repository for version control.

• A webhook triggers the Jenkins pipeline whenever new code is pushed to the repository.

• Jenkins pulls the latest version of the code from the GitHub repository.

• The application is built using Maven, which compiles the source code and manages dependencies.

• Unit tests are executed using JUnit to verify the functionality of the calculator operations.

• After a successful build, Jenkins creates a Docker image for the application.

• The Docker image is pushed to Docker Hub for storage and distribution.

• Ansible pulls the Docker image from Docker Hub and deploys the container on the target environment.

• Finally, the scientific calculator application runs inside the Docker container and can be accessed through the terminal.

Architecture Diagram of DevOps Pipeline

Developer

↓

GitHub Repository

↓

Jenkins CI/CD Pipeline

↓

Build & Test (Maven + JUnit)

↓

Docker Image Creation

↓

Docker Hub Registry

↓

Deployment using Ansible

↓

Docker Container Running Application

---

6. Project Implementation

6.1 Scientific Calculator Application

The Scientific Calculator is implemented as a menu-driven command line interface (CLI) application. The user selects an operation from the menu and provides the required input values to perform the calculation. The program then processes the input and displays the result in the terminal.

The application supports the following mathematical operations:

- Addition
- Subtraction
- Multiplication
- Division
- Percentage Calculation
- Square Root
- Factorial
- Natural Logarithm
- Power Function

```
Scientific Calculator
1. Add
2. Subtract
3. Multiply
4. Divide
5. Percentage
6. Square Root
7. Factorial
8. Natural Log
9. Power
10. Sine
11. Cosine
12. Tangent
13. Exit
Enter Choice:
```

Based on the selected operation and the input values provided by the user, the calculator performs the computation and displays the result. The menu-driven interface makes the application simple and easy to use from the terminal.

6.2 Unit Testing using JUnit

JUnit is used to implement unit testing for the calculator functions. Unit tests are written to verify that each mathematical operation of the scientific calculator returns the correct output for given input values.

During the build process, Maven automatically executes the JUnit test cases to validate the functionality of the calculator operations such as square root, factorial, natural logarithm, and power function. If any test fails, the build process stops, ensuring that incorrect code is not deployed.

Unit testing helps in improving the reliability of the application by detecting errors early in the development process and ensuring that all calculator functions behave as expected.

JUnit Test Execution

```
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------
[INFO] Total time:  1.157 s
[INFO] Finished at: 2026-03-07T22:43:57+05:30
[INFO] ------------------------------------------------------------
```

Figure: JUnit test cases executed successfully for scientific calculator operations.

6.3 Maven Build Process

Maven is used as the build automation tool for this project. It is responsible for compiling the Java source code, managing project dependencies, running unit tests, and packaging the application into an executable file.

The project uses the pom.xml file to define dependencies, plugins, and build configurations required for the application.

The following command is used to build the application: **mvn clean package**

The clean command removes previously compiled files, while the package command compiles the source code, executes unit tests, and generates a packaged application file (JAR) in the target directory.

If the build process is successful, Maven displays a BUILD SUCCESS message in the terminal.

Successful Maven Build Output

```
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ calculator ---
[INFO] Building jar: /home/abhijeet/Desktop/calculator/target/calculator-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.247 s
[INFO] Finished at: 2026-03-07T22:48:28+05:30
[INFO] ------------------------------------------------------------------------
```

Figure: Successful Maven build showing application packaging.

6.4 Jenkins CI/CD Pipeline

Jenkins is used to automate the Continuous Integration and Continuous Deployment (CI/CD) pipeline for the scientific calculator application. Whenever the source code is pushed to the GitHub repository, a webhook automatically triggers the Jenkins pipeline.

The pipeline executes a sequence of automated steps to build, test, and deploy the application. This automation ensures faster and more reliable software delivery while reducing manual effort.

The Jenkins pipeline performs the following tasks:

- Checkout the latest source code from the GitHub repository.
- Build the application using Maven.
- Execute unit tests using JUnit to verify the functionality of the calculator operations.
- Build a Docker image for the application.
- Push the Docker image to Docker Hub.
- Deploy the application container using Ansible.

6.4.1 GitHub Repository Integration

The source code of the scientific calculator application is stored in a GitHub repository. Jenkins is configured to access this repository and retrieve the latest code whenever the pipeline is triggered.

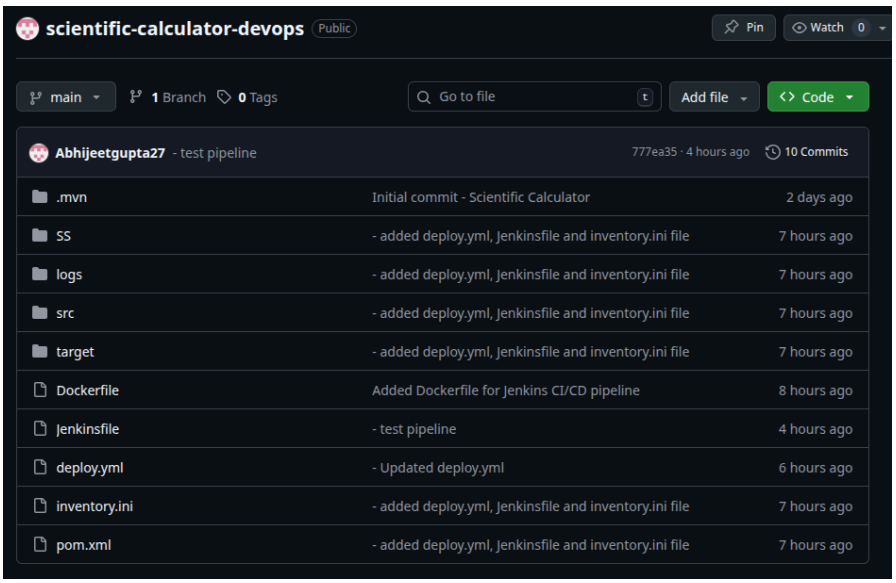


Figure: GitHub repository containing the scientific calculator project source code.

6.4.2 GitHub Webhook Configuration

A webhook is configured in the GitHub repository to automatically trigger the Jenkins pipeline whenever new code is pushed. This enables automatic execution of the CI/CD pipeline without manual intervention.

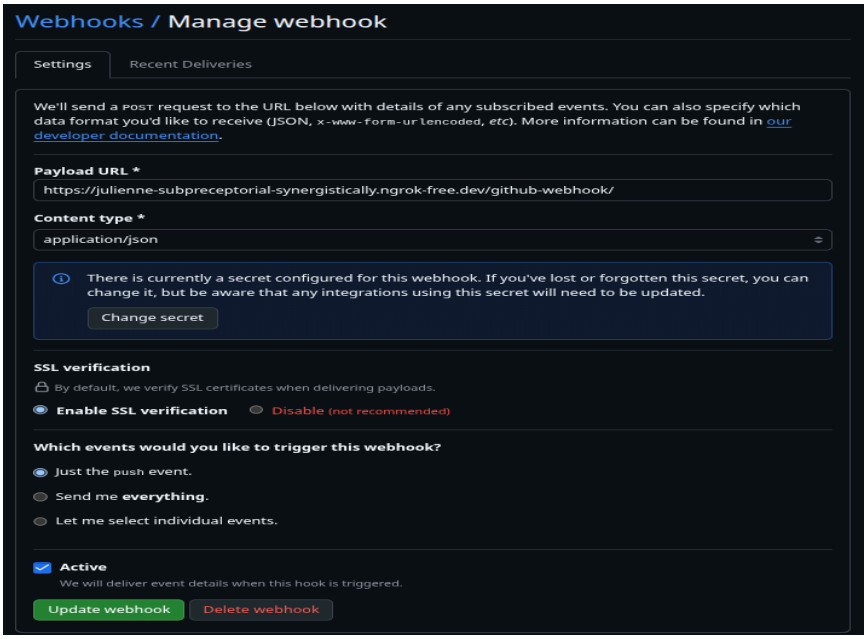


Figure: GitHub webhook configuration used to trigger Jenkins pipeline.

6.4.3 ngrok Tunnel

Since Jenkins is running on a local machine, an ngrok tunnel is used to expose the Jenkins server to the internet. This allows GitHub to communicate with the local Jenkins server and trigger the pipeline through the webhook.

```
ngrok                                                                    (Ctrl+C to quit)

One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status                online
Account                       Abhijeet Kumar Gupta (Plan: Free)
Update                        update available (version 3.37.1, Ctrl-U to update)
Version                       3.35.0
Region                        India (in)
Latency                       20ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://julienne-subpreceptorial-synergistically.ngrok-free.dev -> http://l

Connections                   ttl     opn     rt1     rt5     p50     p90
                              2       0       0.00    0.00    30.72   31.42

HTTP Requests
-------------

19:28:39.735 IST POST /github-webhook/          200 OK
18:50:48.067 IST POST /github-webhook/          200 OK
```

Figure: ngrok tunnel used to expose the local Jenkins server for webhook communication.

6.4.4 Jenkins Dashboard

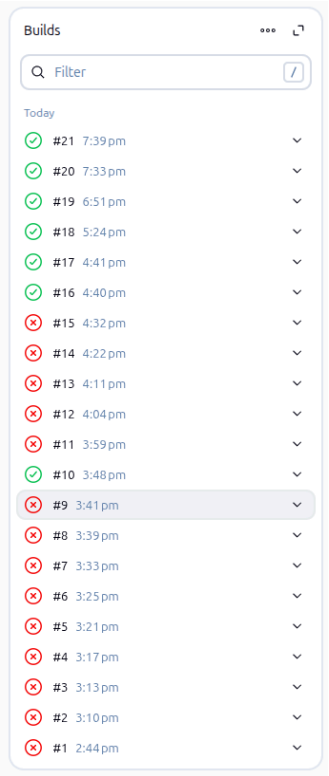
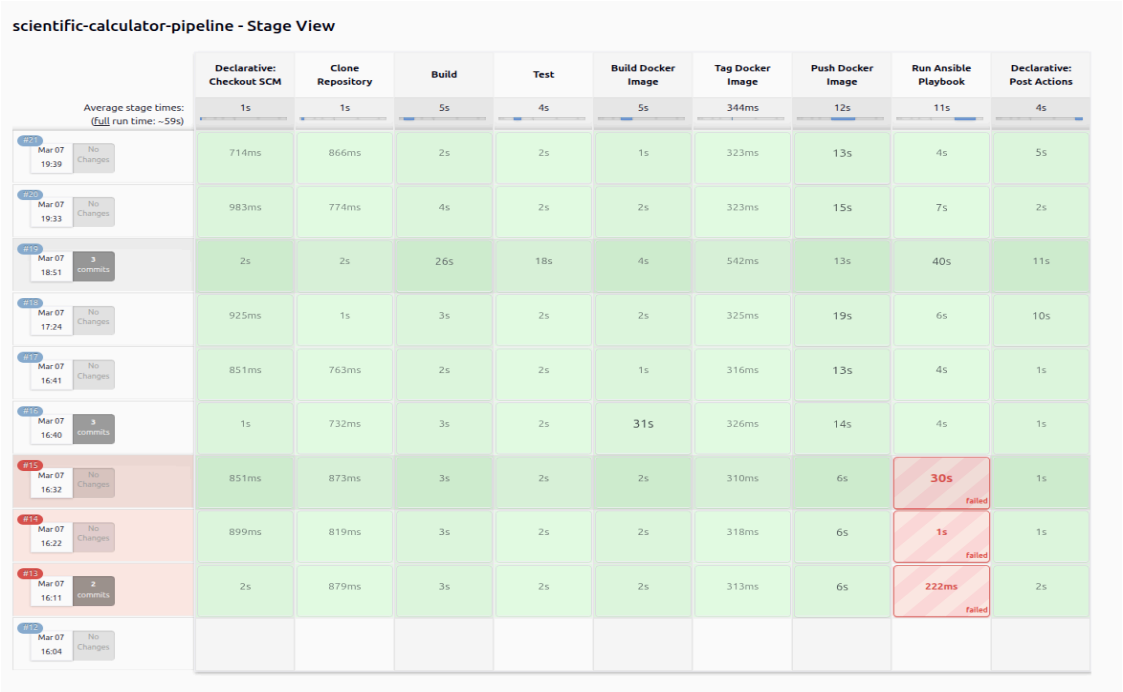


Figure: Jenkins dashboard displaying the configured CI/CD pipeline job.

6.4.5 Jenkins Pipeline Stage View



scientific-calculator-pipeline - Stage View

	Declarative: Checkout SCM	Clone Repository	Build	Test	Build Docker Image	Tag Docker Image	Push Docker Image	Run Ansible Playbook	Declarative: Post Actions
Average stage times: (Full run time: ~59s)	1s	1s	5s	4s	5s	344ms	12s	11s	4s
#21 Mar 07 19:39 No Changes	714ms	866ms	2s	2s	1s	323ms	13s	4s	5s
#20 Mar 07 19:33 No Changes	983ms	774ms	4s	2s	2s	323ms	15s	7s	2s
#19 Mar 07 18:51 3 commits	2s	2s	26s	18s	4s	542ms	13s	40s	11s
#18 Mar 07 17:24 No Changes	925ms	1s	3s	2s	2s	325ms	19s	6s	10s
#17 Mar 07 16:41 No Changes	851ms	763ms	2s	2s	1s	316ms	13s	4s	1s
#16 Mar 07 16:40 3 commits	1s	732ms	3s	2s	31s	326ms	14s	4s	1s
#15 Mar 07 16:32 No Changes	851ms	873ms	3s	2s	2s	310ms	6s	30s failed	1s
#14 Mar 07 16:22 No Changes	899ms	819ms	3s	2s	2s	318ms	6s	1s failed	1s
#13 Mar 07 16:11 2 commits	2s	879ms	3s	2s	2s	313ms	6s	222ms failed	2s
#12 Mar 07 16:04 No Changes									

Figure: Jenkins pipeline stage view showing successful execution of all pipeline stages including build, test, Docker image creation, and deployment.

6.4.6 Jenkins Email Notification

Jenkins is configured to send email notifications after the completion of the pipeline execution. This helps developers monitor the build status without manually checking the Jenkins dashboard.

Whenever the pipeline finishes execution, Jenkins automatically sends an email notification indicating whether the build was successful or failed. This improves monitoring and allows quick identification of any issues in the CI/CD pipeline.

Email Notification Received

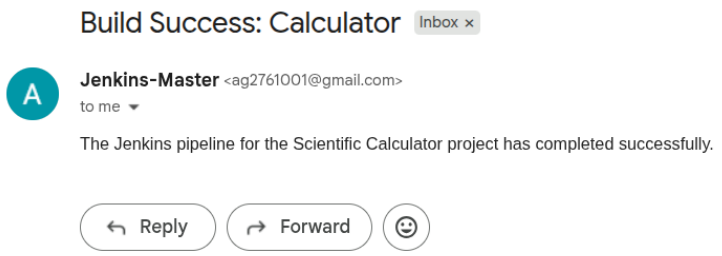


Figure: Email notification sent by Jenkins after successful pipeline execution.

6.5 Docker Containerization

Docker is used to containerize the scientific calculator application so that it can run consistently across different environments. Containerization ensures that the application and all its dependencies are packaged together, making deployment easier and more reliable.

A Dockerfile is used to define the environment required to run the application. It specifies the base image, required dependencies, and commands needed to build and run the application inside a container.

The Docker image for the application is built using the following command:

```
docker build -t scientific-calculator .
```

This command creates a Docker image named scientific-calculator, which contains the compiled application and the runtime environment needed to execute it.

Dockerfile Configuration

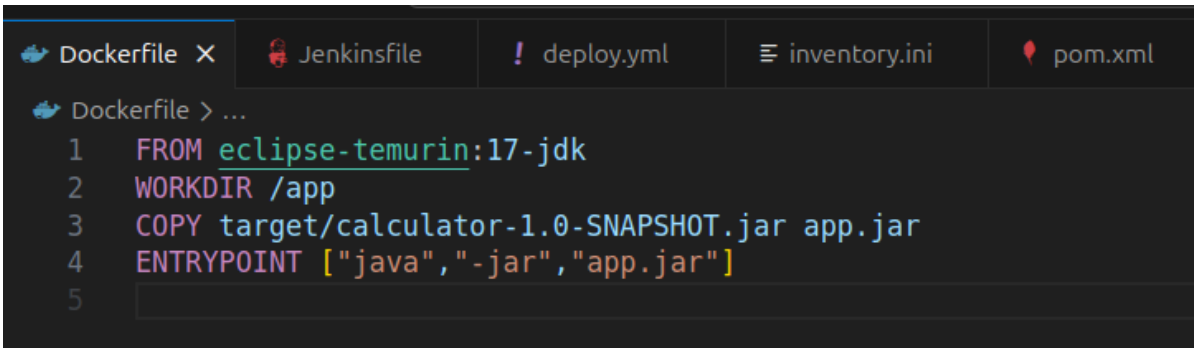


Figure: Dockerfile used to define the container environment for the scientific calculator application.

6.6 DockerHub Image Registry

Docker Hub is used as the container registry to store and manage Docker images for the scientific calculator application. After the Docker image is created, it is pushed to Docker Hub so that it can be accessed and deployed from any system.

Storing the image in Docker Hub allows the CI/CD pipeline to pull the latest version of the application image during deployment.

The Docker image is pushed to Docker Hub using the following command:

docker push <dockerhub-username>/scientific-calculator

Once the image is uploaded successfully, it becomes available in the Docker Hub repository and can be pulled using the docker pull command on any machine.

DockerHub Repository

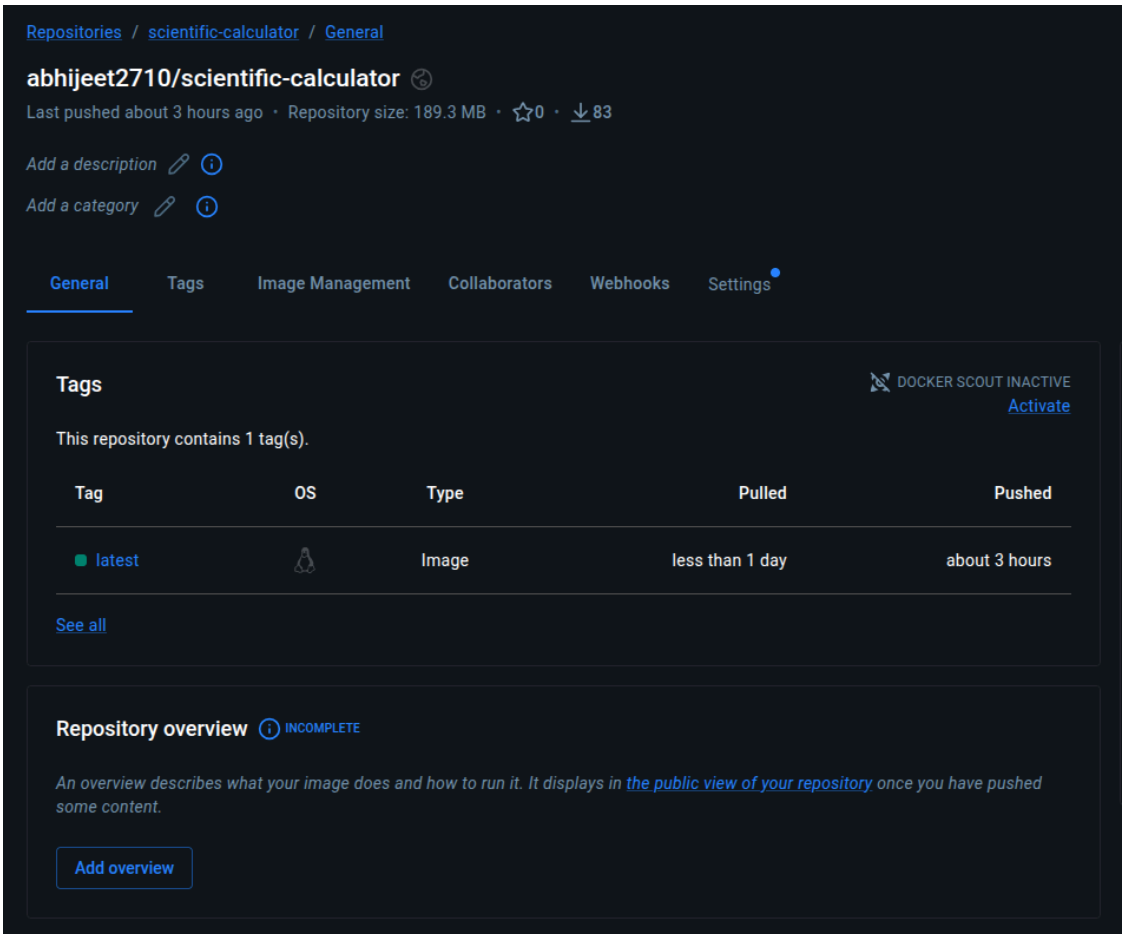


Figure: Docker Hub repository showing the uploaded scientific calculator Docker image.

6.7 Deployment using Ansible

Ansible is used to automate the deployment of the Docker container for the scientific calculator application. It simplifies the deployment process by executing predefined tasks on the target machine using an Ansible playbook.

The deployment configuration is written in a playbook file named deploy.yml, which contains a sequence of tasks required to set up the environment and run the application container.

The Ansible playbook performs the following tasks:

- Installs Docker on the target machine.
- Starts the Docker service.
- Pulls the latest Docker image from Docker Hub.
- Runs the Docker container for the scientific calculator application.

By using Ansible, the deployment process becomes automated, repeatable, and consistent across different systems.

Ansible Playbook (deploy.yml)

```yaml
deploy.yml
---
- name: Deploy Calculator
  hosts: localhost
  become: yes # Enable sudo
  become_user: root # Run tasks as root
  tasks:
    - name: Install Docker
      apt:
        name: docker.io
        state: present

    - name: Start Docker service
      service:
        name: docker
        state: started

    - name: Pull Docker image
      docker_image:
        name: abhijeet2710/scientific-calculator
        source: pull

    - name: Run container
      docker_container:
        name: calculator
        image: abhijeet2710/scientific-calculator
        state: started
        restart_policy: always
        interactive: true
```

Figure: Ansible playbook used for automated deployment of the scientific calculator Docker container.

---

7. CI/CD Pipeline Workflow

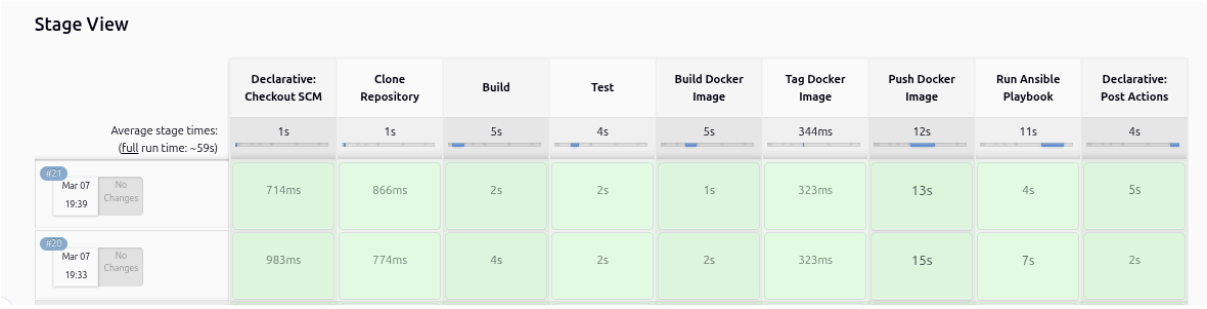
The CI/CD pipeline automates the complete software development lifecycle of the scientific calculator application. It integrates different DevOps tools to automatically build, test, containerize, and deploy the application whenever code changes are made.

Whenever a developer pushes code to the GitHub repository, the following automated workflow is executed:

- GitHub triggers the Jenkins pipeline using a webhook.
- Jenkins checks out the latest source code from the GitHub repository.
- Maven builds the project and runs the unit tests.
- A Docker image of the application is created.
- The Docker image is pushed to Docker Hub.
- Ansible pulls the Docker image and deploys the container.

This automated process ensures that every code change is automatically built, tested, and deployed, which improves reliability and reduces manual effort in software delivery.

Successful Jenkins Pipeline Execution



Stage View

	Declarative: Checkout SCM	Clone Repository	Build	Test	Build Docker Image	Tag Docker Image	Push Docker Image	Run Ansible Playbook	Declarative: Post Actions
Average stage times: (full run time: ~59s)	1s	1s	5s	4s	5s	344ms	12s	11s	4s
#21 Mar 07 19:39 No Changes	714ms	866ms	2s	2s	1s	323ms	13s	4s	5s
#20 Mar 07 19:33 No Changes	983ms	774ms	4s	2s	2s	323ms	15s	7s	2s

Figure: Successful execution of the CI/CD pipeline in Jenkins showing build, test, Docker image creation, and deployment stages.

---

8. Application Output

After successful deployment, the scientific calculator application runs inside a Docker container. The container can be executed using the following command:

docker run -it <dockerhub-username>/scientific-calculator

Once the container starts, the calculator menu appears in the terminal, allowing users to select different mathematical operations and provide input values interactively.

Running Docker Container

```
23:33:43 abhijeet@abhijeet-Inspiron-3501:~/Desktop/calculator 😎 $ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORT
S     NAMES
a11de114d375   abhijeet2710/scientific-calculator   "java -jar app.jar"   4 hours ago   Up 4 hours
      calculator
```

Figure: Docker container running the scientific calculator application.

Calculator Running in Terminal

```
23:11:38 abhijeet@abhijeet-Inspiron-3501:~/Desktop
/calculator 😎 $ docker run -it abhijeet2710/scien
tific-calculator


Scientific Calculator
1. Add
2. Subtract
3. Multiply
4. Divide
5. Percentage
6. Square Root
7. Factorial
8. Natural Log
9. Power
10. Sine
11. Cosine
12. Tangent
13. Exit
Enter Choice:
```

Figure: Scientific calculator application running inside a Docker container in the terminal.

8.1 Square Root Operation

```
Scientific Calculator
1. Add
2. Subtract
3. Multiply
4. Divide
5. Percentage
6. Square Root
7. Factorial
8. Natural Log
9. Power
10. Sine
11. Cosine
12. Tangent
13. Exit
Enter Choice: 6
Enter number: 9
Result: 3.0
```

Figure: Square root calculation performed using the scientific calculator.

8.2 Factorial Operation

```
Scientific Calculator
1. Add
2. Subtract
3. Multiply
4. Divide
5. Percentage
6. Square Root
7. Factorial
8. Natural Log
9. Power
10. Sine
11. Cosine
12. Tangent
13. Exit
Enter Choice: 7
Enter number: 5
Result: 120
```

Figure: Factorial calculation performed using the scientific calculator.

8.3 Power Operation

```
Scientific Calculator
1. Add
2. Subtract
3. Multiply
4. Divide
5. Percentage
6. Square Root
7. Factorial
8. Natural Log
9. Power
10. Sine
11. Cosine
12. Tangent
13. Exit
Enter Choice: 9
Enter base: 2
Enter power: 5
Result: 32.0
```

Figure: Power function calculation performed using the scientific calculator.

8.4 Natural Log

```
Scientific Calculator
1. Add
2. Subtract
3. Multiply
4. Divide
5. Percentage
6. Square Root
7. Factorial
8. Natural Log
9. Power
10. Sine
11. Cosine
12. Tangent
13. Exit
Enter Choice: 8
Enter number: 10
Result: 2.302585092994046
```

Figure: Natural Log function calculation performed using the scientific calculator.

---

10. Final Result

The project successfully demonstrates the implementation of a complete DevOps CI/CD pipeline for a scientific calculator application. The application is developed as a command-line interface (CLI) program that performs several mathematical operations such as square root, factorial, natural logarithm, and power functions.

The DevOps pipeline automates the entire software development lifecycle including source code management, building, testing, containerization, and deployment. Whenever code is pushed to the GitHub repository, Jenkins automatically triggers the pipeline to build the application using Maven, run unit tests, create a Docker image, push the image to Docker Hub, and deploy the container using Ansible.

By integrating multiple DevOps tools such as GitHub, Jenkins, Maven, Docker, Docker Hub, and Ansible, the system ensures faster, reliable, and automated software delivery. This project demonstrates how DevOps practices can improve development efficiency, reduce manual effort, and ensure consistent deployment across different environments.

---

11. Project Links

The following links provide access to the source code and Docker image repository for the scientific calculator DevOps project.

GitHub Repository
This repository contains the complete source code of the scientific calculator application along with configuration files for the DevOps pipeline.

Link: https://github.com/Abhijeetgupta27/scientific-calculator-devops.git

DockerHub Repository
This repository stores the Docker image of the scientific calculator application, which can be pulled and executed from any system using Docker.

Link: https://hub.docker.com/repository/docker/abhijeet2710/scientific-calculator/general

---

12. Conclusion

This project demonstrates how DevOps practices can be applied to automate the software development and deployment processes. A scientific calculator application was developed and integrated with a complete CI/CD pipeline to automate the build, testing, containerization, and deployment stages.

By integrating DevOps tools such as GitHub, Maven, Jenkins, Docker, Docker Hub, and Ansible, the entire application lifecycle becomes faster, more reliable, and easier to manage. The CI/CD pipeline ensures that whenever new code is pushed to the repository, it is automatically built, tested, and deployed without manual intervention.

This automated approach improves software quality, reduces manual effort, and ensures consistent deployment across different environments. Overall, the project highlights the importance of DevOps in modern software development for achieving efficient and reliable software delivery.